

VOICEBUDDY

Technical Deep Dive

Latency Architecture • Turn-Taking & Barge-In • Service Selection • Open Questions Resolved

February 2026 • POC Planning Document

What This Document Decides

This document resolves four open questions from the initial VoiceBuddy plan and dives deep into the three areas that will determine whether the POC succeeds or fails: **latency architecture**, turn-taking and interruption handling, and service selection. It is grounded in current benchmarks and real-world data from the providers, not marketing claims.

Bottom line: a chained pipeline (Deepgram Flux → Claude Haiku/Sonnet → Cartesia Sonic) with eager end-of-turn speculative firing and a dual-layer LLM strategy can realistically hit 700–900 ms end-to-end latency. That is the sweet spot — human-perceptible as responsive, achievable without exotic infrastructure.

1. Latency Architecture — The Budget & How to Hit It

Every millisecond in a voice pipeline is earned, not given. This section breaks down exactly where time goes, what the realistic numbers are based on current provider benchmarks, and the specific technique that buys us the most back.

1.1 The Latency Budget

Human conversational response time is 200–400 ms. Anything above 800 ms starts feeling like a pause. Anything above 1.2 seconds feels robotic. Our target is to land the "realistic" column consistently below 1,000 ms.

Pipeline Stage	Best Case	Realistic	Worst Case	Notes
1. End-of-Speech Detection	150 ms	250 ms	400 ms	Flux EOT
2. Final Transcript Return	50 ms	100 ms	200 ms	Deepgram Flux
3. LLM First Token	100 ms	250 ms	500 ms	Haiku vs Sonnet
4. TTS First Audio Byte	40 ms	90 ms	200 ms	Cartesia Sonic
5. Network + Playback	30 ms	80 ms	150 ms	Geo-dependent
TOTAL END-TO-END	370 ms	770 ms	1,450 ms	Target: < 1,000 ms

Stage-by-Stage Breakdown

- Stage 1 — End-of-Speech Detection: This is the hidden killer. Most voice apps sound robotic not because of slow LLMs but because they wait too long to decide the user finished talking. Deepgram Flux is the first STT model with turn-taking built directly into the recognition model — it emits structured events (StartOfTurn, EndOfTurn) rather than requiring a separate VAD layer stitched on top. At default thresholds, Flux delivers end-of-turn detection at roughly 260 ms (p50). That is the baseline we plan against.
- Stage 2 — Final Transcript: Once Flux fires EndOfTurn, the final transcript is already available or arrives within 50–100 ms. This is not a bottleneck.
- Stage 3 — LLM First Token: This is the most variable stage and depends heavily on model choice. Claude Haiku 4.5 consistently delivers first tokens in the 100–200 ms range. Claude Sonnet 4.5 is slower (200–300 ms) but produces significantly more natural conversational responses. The dual-layer strategy in Section 1.3 addresses this trade-off directly.
- Stage 4 — TTS First Audio Byte: Cartesia Sonic 3 is purpose-built for this. Its model latency sits at 40–90 ms, which is the fastest in production TTS. ElevenLabs Flash 2.5 is close (75–100 ms) but Cartesia's edge here is real and measurable.
- Stage 5 — Network + Playback: This is largely outside our control but is minimized by co-locating services and using WebSocket streaming rather than HTTP round-trips.

1.2 The Speculative Firing Technique (Eager End-of-Turn)

This is the single most impactful optimization available to us. Deepgram Flux introduced "Eager End-of-Turn" — a medium-confidence signal that the user is probably done speaking, fired 150–250 ms before the high-confidence EndOfTurn event. The idea: start the LLM call speculatively on that medium-confidence signal. If the user keeps talking, a TurnResumed event arrives and we cancel the draft. If they actually stopped, we already have a head start on the LLM response.

Standard Flow	Eager EOT Flow	Savings
User finishes speaking	User finishes speaking	—
Wait for EndOfTurn (~260ms)	EagerEndOfTurn fires (~100ms)	~160 ms saved
Send transcript to LLM	LLM call already in flight	LLM TTFT overlapped
Wait for first token	First token may already exist	0–250 ms saved
Send to TTS	Send to TTS immediately	—
Total overhead	Speculative overhead	150–400 ms saved

The cost: 50–70% more LLM API calls because some will be wasted on cancelled drafts. For a POC this is fine. At scale, the cost/latency trade-off needs to be tuned based on actual cancellation rates (Deepgram recommends tracking EagerEndOfTurn → TurnResumed ratio to calibrate).

POC recommendation: Start with EndOfTurn-only (simpler, fewer LLM calls). Once the baseline is measured and stable, layer in EagerEndOfTurn and compare. The architecture should support both modes from Day 1 so the switch is a config change, not a rewrite.

1.3 The Dual-Layer LLM Strategy

The core tension: Haiku is fast but less nuanced. Sonnet is natural but slower. For a product where the output is spoken aloud, naturalness matters enormously — but so does not making the caller wait. The solution is two layers, not two models competing.

Metric	Claude Haiku 4.5	Claude Sonnet 4.5
First Token Latency	~100 ms	~200–300 ms
Conversational Quality	Good — concise	Excellent — natural
Voice Prompt Following	Strong	Strongest
Cost per Token	Low	Medium
Context Window	200K tokens	1M tokens (beta)
POC Role	Filler responses + simple intents	Complex reasoning + full replies

- Layer 1 — Filler / Acknowledgment (Haiku): While Sonnet is generating the real response, Haiku fires immediately to produce a filler like "Sure, let me look into that" or "Got it, one moment." This is a 10–20 token generation — sub-150 ms. The caller hears something human-like instantly, and the perceived latency drops dramatically even if the full answer takes another 300–500 ms to arrive.

- Layer 2 — Full Response (Sonnet): Generates the actual answer. Because the filler already bought 200–400 ms of perceived time, Sonnet's slightly higher TTFT is no longer a bottleneck. Its responses also stream sentence-by-sentence into TTS, so playback begins before the full response is complete.

Prompt caching on Anthropic's API is critical here. The system prompt (business context, conversation rules, persona) stays constant across a session. Caching it means subsequent calls within the same conversation pay only for the new tokens, cutting TTFT significantly after the first turn.

2. Turn-Taking & Barge-In — Making It Feel Human

This is where most voice AI products break. Low latency means nothing if the bot talks over the user or sits in awkward silence. This section defines the state machine that governs when VoiceBuddy listens, thinks, speaks, and stops.

2.1 The Conversation State Machine

The system is always in exactly one state. Transitions are triggered by audio events (VAD, Flux turn events) or system events (LLM response ready, TTS complete). Every state has exactly one set of rules. No ambiguity.

State	Audio Input	System Output	Action
IDLE	Silence / noise	Nothing	VAD monitoring ON
USER_SPEAKING	Speech detected	Nothing	Stream to STT
PROCESSING	Silence	Nothing	LLM generating
BOT_SPEAKING	Silence / noise	TTS audio playing	VAD still ON
BARGE-IN DETECTED	Speech during BOT_SPEAKING	TTS STOPS immediately	Cancel LLM + TTS
FILLER RESPONSE	Silence (< 500ms)	"Sure..." / "Got it..."	Buy time for LLM

2.2 Barge-In (Interruption) Architecture

Barge-in is the hardest problem in voice AI. The system must monitor for user speech continuously — including while it is playing its own audio back. This creates a technical challenge: the bot's own audio can trigger its own microphone (echo), which looks like user speech to a naive VAD.

The Three-Layer Detection Stack

- Layer 1 — Echo Cancellation: WebRTC's built-in Acoustic Echo Cancellation (AEC) handles the browser case. For phone (Twilio), we need an adaptive echo canceller in the server pipeline. This runs before any VAD or STT sees the audio. Without it, the bot will constantly interrupt itself.
- Layer 2 — Voice Activity Detection (VAD): Silero VAD is the industry standard. It processes audio in 10–20 ms frames and outputs a speech/no-speech probability. It runs independently of STT and is always on. Critical config: the sensitivity threshold. Too high → misses real interruptions. Too low → background noise triggers false barge-ins.
- Layer 3 — Confirmation via STT: A VAD hit alone is not enough to trigger a barge-in. Background noise, coughs, or short interjections like "uh-huh" should not necessarily stop the bot mid-sentence. The STT layer (Deepgram Flux) confirms whether actual speech content is present. Flux's StartOfTurn event is the signal we act on — it means the model is confident that a new conversational turn has begun.

What Happens on Barge-In Detection

- Stop TTS playback immediately — not at the end of the current sentence. Mid-sentence. This is non-negotiable for naturalness.

- Cancel the in-flight LLM generation if it hasn't finished. No point spending tokens on a response that will never be heard.
- Clear the TTS audio buffer. Any queued-but-not-yet-played audio is discarded.
- Transition to USER_SPEAKING state. The new utterance is processed as a fresh turn with full context of what was said before the interruption.

2.3 Silence & Filler Handling

Silence in a phone call is not the same as silence in a chat. A 2-second pause in a phone call is uncomfortable. A 5-second pause is alarming. VoiceBuddy needs a silence policy.

- 0–500 ms silence after user stops speaking: Normal processing time. No action needed. The filler system (Section 1.3) fires here if the LLM hasn't returned yet.
- 500 ms–2 s silence: The bot is thinking. A filler should have already played. If not, inject one now: "Just a moment..."
- 2–5 s silence: Something is wrong. Either the LLM timed out or the user got distracted. Prompt gently: "Are you still there?"
- 5+ s silence: Assume the call may be dropping. One more prompt, then graceful close: "It seems we may have lost the connection. I'll follow up with you shortly. Have a great day."

2.4 The Backchanneling Problem

Humans constantly signal "I'm listening" while the other person talks: "Mm-hmm," "Right," "I see." These are called backchannels. A voice agent that never does this sounds cold and robotic. But a voice agent that does it at the wrong time sounds broken.

POC approach: Do not attempt real-time backchanneling in the POC — it requires audio-level timing precision that is very hard to get right. Instead, build it into the LLM response pattern. When the user gives a long explanation, the LLM's first sentence should be an acknowledgment ("Got it, so you're saying...") before jumping to the solution. This achieves the same psychological effect through the response content, not through mid-stream audio injection.

3. Service Selection — Decisions With Rationale

3.1 Speech-to-Text: Deepgram Flux

Decision: Deepgram Flux is the STT for VoiceBuddy POC. Not Nova-3. Not a competitor.

- Flux is purpose-built for conversational turn-taking. It replaces the entire STT + VAD + endpointing stack with a single model that emits structured turn events. Independent benchmarking by Coval confirmed Flux delivers the fastest time-to-first-token among all providers tested, with competitors trailing by 0.5–1.5 seconds at P50.
- Flux matches Nova-3 on transcription accuracy (WER) while adding conversational intelligence. You do not sacrifice accuracy for speed.
- The Eager End-of-Turn feature (Section 1.2) is Flux-exclusive. This is the speculative firing mechanism that buys us 150–400 ms.
- Flux requires the /v2/listen endpoint with 80 ms chunk sizes. This is a config detail, not a complexity concern.
- One limitation: Flux is currently English-only. For international verticals, Nova-3 (multilingual) becomes relevant. Not a POC blocker.

3.2 Text-to-Speech: Cartesia Sonic 3 (Primary) + ElevenLabs Flash (Fallback)

Metric	Cartesia Sonic 3	ElevenLabs Flash 2.5	Deepgram Aura-2
Model Latency (TTFA)	~40–90 ms	~75–100 ms	< 200 ms
Voice Quality	Very Good	Best (industry leader)	Good
Voice Cloning	Yes (10s audio)	Yes (Professional)	No
Languages	15	70+	2
Streaming WebSocket	Yes	Yes	Yes
Relative Cost	~\$\$\$	~\$\$\$\$\$	~\$\$
POC Verdict	PRIMARY — lowest latency	FALLBACK — best quality	SKIP for now

Why Cartesia first, not ElevenLabs: ElevenLabs has better voice quality, but for a phone-based product, latency is more perceptible than subtle audio quality differences. A caller doesn't compare voice naturalness side-by-side — they notice whether there's a pause before the bot starts talking. Cartesia's 40–90 ms TTFA vs ElevenLabs' 75–100 ms is a real edge when every stage of the pipeline is stacking.

Why not Deepgram Aura-2: Aura-2 is a solid enterprise option with good latency and simple pricing, but it supports only 2 languages and has no voice cloning. VoiceBuddy's value proposition includes letting each business have a personalized voice — cloning is a key differentiator.

Sentence-level streaming is critical: We do not wait for the full LLM response before sending to TTS. Each complete sentence is sent as it streams out of the LLM. Cartesia and ElevenLabs both support this. Splitting mid-sentence produces choppy audio — the chunking boundary must be at punctuation.

3.3 LLM: Claude (Anthropic) — Haiku + Sonnet Dual Stack

Claude's advantage for voice is not just quality — it is instruction-following behavior in conversational contexts. **Voice prompting is a craft problem:** responses must be short-sentence, conversational, never lecture-y. Claude follows these constraints more reliably than alternatives in testing. The dual-layer strategy (Section 1.3) is the recommended architecture.

4. Open Questions — All Resolved

4.1 Single WebSocket vs. Separate Connections Per Stage?

Single Persistent WebSocket	Separate Connections Per Stage
Lower raw latency (one hop)	Each service has its own WebSocket
Tightly coupled — if one service changes protocol, everything breaks	Swappable — change STT without touching TTS
Hard to debug which stage is slow	Each connection is independently monitorable
Vendor lock-in risk is high	Provider-agnostic architecture

Decision: Separate persistent WebSockets, one per service. The latency difference between one hop and three hops over WebSocket is 20–50 ms — not enough to justify the coupling risk. The chained pipeline is the industry standard for production telephony voice agents precisely because it survives vendor changes and is debuggable. Phone networks already add 100–200 ms of fixed latency that dwarfs this difference.

4.2 Where Does the Brain Live — Client or Server?

Decision: Server. The client is dumb audio I/O only. The browser POC can get away with some client-side logic, but the architecture must be server-first from Day 1. When phone integration arrives (Twilio streams audio to a server WebSocket), there is no client to run logic on. Everything — VAD monitoring, barge-in detection, LLM orchestration, TTS streaming — lives on the server. The browser just captures mic audio and plays back what the server sends.

4.3 Conversation State Structure — What Do We Actually Store?

Decision: In-memory session store with Redis for POC scale. In-memory alone fails if the server restarts mid-call. Redis with a short TTL (call duration + buffer) gives persistence without a database. Here is exactly what each session stores:

Field	Type	Purpose
session_id	UUID	Unique per call
transcript_history	Array<{role, text, timestamp}>	Full conversation for LLM context
confidence_scores	Array<Float>	Per-utterance STT confidence — for debugging
latency_log	Array<{stage, ms, timestamp}>	Latency measurement at every stage
conversation_state	Enum (IDLE, USER_SPEAKING, etc.)	Current state machine position
business_context	JSON	The client business's info, hours, services

audio_segments	Array<Blob ref>	Raw audio for debugging — optional, expensive
-----------------------	-----------------	---

4.4 Echo Cancellation — How Do We Prevent the Bot From Hearing Itself?

Decision: WebRTC AEC for browser; server-side adaptive echo canceller for phone. In the browser, WebRTC's built-in Acoustic Echo Cancellation handles this automatically — it's one of the reasons WebRTC is the right transport for the POC. When we move to phone (Twilio), the audio arrives as a raw stream with no AEC. We need to run an adaptive echo canceller on the server before the audio hits Deepgram. RNNoise or similar lightweight models can handle this with minimal latency overhead.

5. Cost Model — Is This Viable?

Unit economics matter even at POC stage. If the per-minute cost makes the product unviable at any scale, we need to know now. These estimates are based on a typical 3–5 minute inbound call with moderate back-and-forth conversation.

Service	Model	Est. \$/hr	Notes
STT — Deepgram	Flux	\$0.015	Streaming
LLM — Anthropic	Haiku 4.5 (filler) + Sonnet 4.5 (main)	\$0.08–0.40	Blended
TTS — Cartesia	Sonic 3	\$0.10–0.20	Streaming chars
Telephony (later)	Twilio	\$0.015	Per min + phone #
ESTIMATED TOTAL		\$0.21–0.65/hr	~\$0.004–0.011/min

Verdict: At \$0.004–0.011 per minute, a 5-minute call costs \$0.02–0.055. If VoiceBuddy charges even \$1–5 per call (or a flat monthly subscription per business line), the unit economics are very healthy. The LLM layer is the largest variable cost — the dual-layer strategy (Haiku for fillers) keeps this in check.

6. POC Success Criteria

These are the specific, measurable outcomes the POC must prove. If any one of these fails, VoiceBuddy does not move to production in its current form.

Hypothesis	Pass Threshold	Measurement Method
End-to-end latency is perceptibly fast	< 1,000 ms at p75	Timestamp every stage, log total
Barge-in works reliably	< 100 ms TTS cutoff after speech detected	Test with scripted interruptions
Conversation stays on-task for structured flows	< 5% off-topic derailment in 50 test calls	Scripted scenario replay
Blind listening test fools listeners	> 40% of listeners unsure if human or AI in first 30s	Record calls, blind evaluation
System degrades gracefully on errors	No crashes in 100 edge-case scenarios	Noise, silence, gibberish, timeouts

Next Steps

- Finalize service accounts: Deepgram (Flux), Anthropic (Haiku + Sonnet), Cartesia (Sonic 3)
- Set up latency measurement harness before writing any voice logic — you cannot optimize what you cannot see
- Build the state machine (Section 2.1) as the first piece of code — everything else plugs into it
- Day 1 target: audio in → audio out, with latency logged at every stage. No LLM yet. Just prove the pipe works
- Voice cloning session: record 30–60 seconds of a "friendly receptionist" voice for Cartesia cloning. This is a Day 1 task — cloning has a processing delay